

Security & Authentication

Authentication, authorization/RBAC, input validation, and production security checklist

Document status: Active

Version: 1.0

Last updated: 2026-09-26

1. Security Objectives

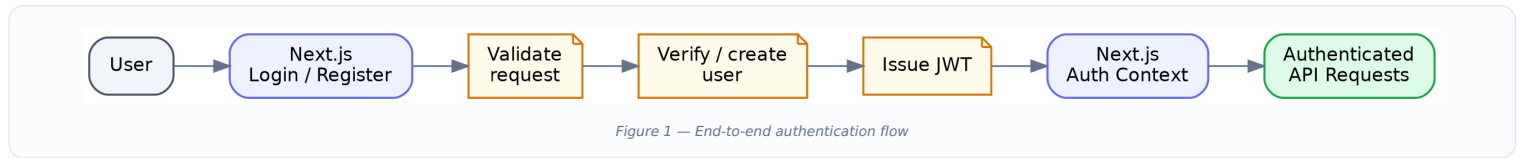
FleetFlow must protect:

- User credentials
- Authentication tokens
- Personal user information
- Fleet information
- Operational requests
- Service records
- Administrative operations
- Environment secrets

2. Authentication

FleetFlow uses JWT-based authentication.

High-level flow:



Authentication must happen on the backend. The frontend must never be considered a trusted security boundary.

3. Password Security

Passwords must:

- Never be stored in plaintext.
- Be hashed using a modern password-hashing algorithm.
- Never be returned in API responses.
- Never be logged.
- Never be committed to source control.

Password requirements should be validated consistently during registration and password changes.

4. JWT Security

JWT configuration must be stored in environment variables.

Example:

```
JWT_SECRET=<strong-random-secret>
```

Rules:

- Never hard-code JWT secrets.
- Never commit `.env` files containing secrets.
- Keep token expiry appropriate for the application's risk profile.
- Validate token signature and expiry on protected endpoints.
- Include only necessary claims.

Typical claims may include:

```
{
  "id": "user-id",
  "role": "manager",
  "email": "user@example.com"
}
```

The exact production token model may evolve to access/refresh tokens.

5. Authorization / RBAC

FleetFlow uses role-based access control.

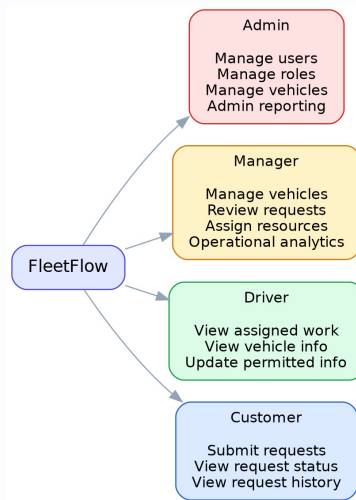


Figure 2 — FleetFlow role model

Authorization must be enforced at the backend.

Example:

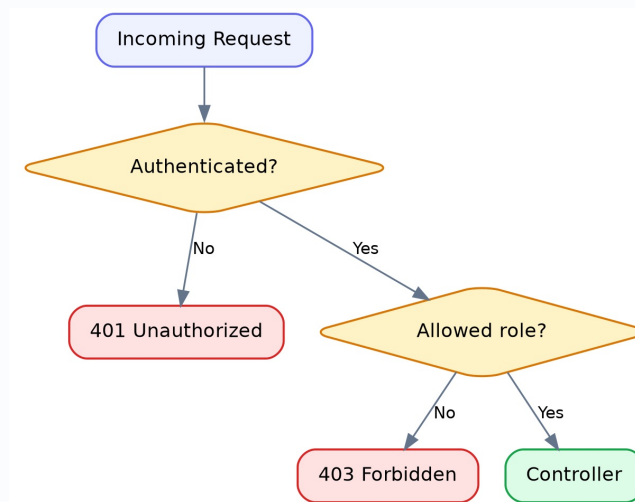


Figure 3 — Authorization decision flow

Do not rely on hiding a button or route in Next.js as authorization.

6. Input Validation

All external input should be validated.

Validation should cover:

- Request bodies
- Route parameters
- Query parameters
- Authentication data
- Vehicle data
- Request data
- Service/history records

Zod is used/expected for structured validation where applicable.

Invalid input should be rejected before business logic executes.

7. Environment Variables

Sensitive configuration belongs in environment variables.

Examples:

```

MONGODB_URI
JWT_SECRET
AUTH_SERVICE_URL
VEHICLE_SERVICE_URL
NEXT_PUBLIC_API_URL
  
```

Never commit:

```
.env
.env.local
production secrets
private keys
database credentials
API keys
```

Use `.env.example` with placeholder values.

8. CORS

CORS should allow only trusted frontend origins in production.

Development may use:

```
http://localhost:3000
```

Production configuration should explicitly define the deployed frontend origin.

Avoid:

```
Access-Control-Allow-Origin: *
```

for authenticated production APIs unless there is a specific, reviewed reason.

9. API Security

Protected APIs should:

- Authenticate requests.
- Authorize operations.
- Validate inputs.
- Use HTTPS in production.
- Return safe errors.
- Avoid leaking stack traces.
- Rate-limit sensitive endpoints where appropriate.

Authentication endpoints should receive additional protection against brute-force attempts.

10. Database Security

MongoDB credentials must remain server-side.

Services should:

- Use least-privilege database credentials where possible.
- Validate IDs and input.
- Use indexes for frequent queries.
- Avoid exposing internal database errors.
- Restrict production database network access.

11. Microservice Security

Services should not assume that an internal request is automatically trusted.

As FleetFlow matures:

- Use service authentication for sensitive service-to-service requests.
- Validate service identity.
- Keep database credentials isolated per service where practical.
- Avoid direct database access between services.
- Consider an API gateway for centralized edge concerns.

12. Logging

Logs must not contain:

- Passwords
- JWTs
- API keys
- Database passwords
- Sensitive personal information

Useful logs include:

```
request ID
service
HTTP method
route
status code
duration
non-sensitive error context
```

13. Security Incident Response

If a secret is accidentally committed:

1. Revoke/rotate the exposed secret immediately.

2. Remove it from active configuration.
3. Remove it from repository history where appropriate.
4. Check whether it was accessed.
5. Replace it with a new secret.
6. Review related logs.

Deleting a secret from the latest commit is not sufficient if the secret remains in Git history.

14. Production Security Checklist

- ☐ HTTPS enabled
- ☐ Strong JWT secret configured
- ☐ Production database credentials configured securely
- ☐ CORS restricted
- ☐ Password hashing enabled
- ☐ Request validation enabled
- ☐ RBAC enforced server-side
- ☐ Rate limiting configured
- ☐ Secure logging configured
- ☐ `.env` excluded from Git
- ☐ Secrets rotated after exposure
- ☐ Dependency vulnerabilities reviewed
- ☐ Backups configured
- ☐ Monitoring configured